

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 3 – Konsep Dasar Pemrograman Berorientasi Objek



NAMA: NISA

NIM: 103132400044

KELAS: PS1DS-05-REG-01

PROGRAM STUDI SAINS DATA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2026

A. Dasar Teori

Pemrograman Berorientasi Objek (OOP) adalah metode pemrograman yang membangun sistem menggunakan objek yang berisi data dan fungsi serta dapat saling berinteraksi. Konsep utama OOP meliputi abstraksi, enkapsulasi, inheritance, polymorphism, dan message. Abstraksi berarti mengambil karakteristik dan perilaku penting dari suatu objek. Enkapsulasi adalah penggabungan data dan method sekaligus menyembunyikan detail implementasi melalui data hiding dan modularitas. Inheritance merupakan pewarisan atribut dan method dari superclass kepada subclass. Polymorphism berarti satu interface dapat digunakan untuk menghasilkan tindakan yang berbeda, seperti melalui overloading dan overriding. Sementara itu, message merupakan komunikasi antarobjek melalui pemanggilan method yang dapat disertai parameter.

Dalam OOP, class adalah cetak biru atau rancangan objek, sedangkan object adalah hasil instansiasi dari class. Class memiliki field/atribut untuk menyimpan data dan method untuk menjalankan perilaku atau proses tertentu. Field terdiri dari class field yang menggunakan `static` dan berkaitan dengan class, serta instance field yang dimiliki oleh masing-masing objek. Method dapat mengembalikan nilai menggunakan `return` atau tidak mengembalikan nilai menggunakan `void`.

Dalam pemanggilan method terdapat pass by value dan pass by reference. Pass by value memberikan salinan nilai sehingga perubahan di dalam method tidak memengaruhi variabel asal, sedangkan pass by reference menggunakan referensi objek sehingga perubahan terhadap objek dapat memengaruhi objek asal. Java juga memiliki keyword `this` yang digunakan untuk merujuk pada objek yang sedang digunakan, terutama untuk membedakan atribut dengan parameter yang memiliki nama sama. `this` juga dapat digunakan untuk memanggil constructor lain dalam class yang sama.

B. Guided

a. Abstraksi

kucing.java

```
/*  
  
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change  
 this license
```

```
* Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this
template
```

```
*/
```

```
package com.nisanisaan.project.Modul3.guided.abstraksi;
```

```
/**
```

```
*
```

```
* @author Admin
```

```
*/
```

```
public class kucing {
```

```
    String nama;
```

```
    String ras;
```

```
    void bersuara(){
```

```
        System.out.println(nama + ras + "meow");
```

```
    }
```

```
}
```

Abstraksi.java

```
/*
```

```
* Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change
this license
```

```
* Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this
template
```

```
*/
```

```
package com.nisanisaan.project.Modul3.guided.abstraksi;
```

```
/**
```

```
*
```

```

* @author Admin
*/

public class Abstaraksi {

    public static void main(String[] args) {

        kucing aren = new kucing();

        aren.nama = "Aren Gentong";

        aren.ras = "Mujaer";

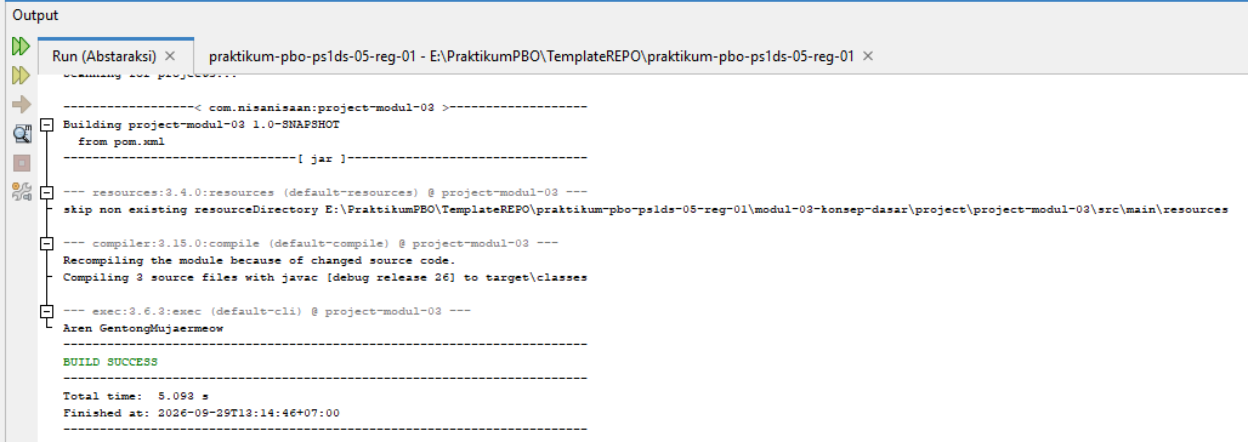
        aren.bersuara();

    }

}

```

Output :



```

Output
Run (Abstaraksi) x praktikum-pbo-ps1ds-05-reg-01 - E:\PraktikumPBO\TemplateREPO\praktikum-pbo-ps1ds-05-reg-01 x
Building project-module-03 1.0-SNAPSHOT
from pom.xml
-----[ jar ]-----
--- resources:3.4.0:resources (default-resources) @ project-module-03 ---
skip non existing resourceDirectory E:\PraktikumPBO\TemplateREPO\praktikum-pbo-ps1ds-05-reg-01\modul-02-konsep-dasar\project\project-module-03\src\main\resources
--- compiler:3.15.0:compile (default-compile) @ project-module-03 ---
Recompiling the module because of changed source code.
Compiling 3 source files with javac [debug release 26] to target\classes
--- exec:3.6.2:exec (default-cli) @ project-module-03 ---
Aren GentongMujaermeow
BUILD SUCCESS
Total time: 5.092 s
Finished at: 2026-09-29T13:14:46+07:00

```

b. Enkapsulasi

Rekening.java

```

/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change
 this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this
 template
 */
package com.nisanisaan.project.Modul3.guided.enkapsulasi;

/**
 *

```

```

* @author Admin
*/
public class Rekening {
    private int saldo = 0;

    public void tambahSaldo(int jumlah){
        saldo = saldo + jumlah;
        System.out.println("saldo berhasil ditambahkan");
    }

    public void tampilkanSaldo(){
        System.out.println("Saldo Anda:" + saldo);
    }
}

```

Enkapsulasi .java

```

/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change
 this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this
 template
 */
package com.nisanisaan.project.Modul3.guided.enkapsulasi;

/**
 *
 * @author Admin
 */
public class Enkapsulasi {

    public static void main(String[] args){
        Rekening rek = new Rekening();

        rek.tambahSaldo(20000000);
        rek.tampilkanSaldo();
    }
}

```

Output :

```
Output
praktikum-pbo-ps1ds-05-reg-01 - E:\PraktikumPBO\TemplateREPO\praktikum-pbo-ps1ds-05-reg-01 x Run (Enkapsulasi) x
-----< com.nisanisaan:project-modul-03 >-----
Building project-modul-03 1.0-SNAPSHOT
from pom.xml
-----[ jar ]-----
--- resources:3.4.0:resources (default-resources) @ project-modul-03 ---
skip non existing resourceDirectory E:\PraktikumPBO\TemplateREPO\praktikum-pbo-ps1ds-05-reg-01\modul-03-konsep-dasar\project\project-modul-03\src\main\resources
--- compiler:3.15.0:compile (default-compile) @ project-modul-03 ---
Recompiling the module because of changed source code.
Compiling 5 source files with javac [debug release 26] to target\classes
--- exec:3.6.3:exec (default-cli) @ project-modul-03 ---
saldo berhasil ditambahkan
Saldo Anda:20000000
-----
BUILD SUCCESS
-----
Total time: 1.690 s
Finished at: 2026-09-29T13:22:19+07:00
-----
```

c. Class

Circle01.java

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change
this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this
template
 */
package com.nisanisaan.project.Modul3.guided;

/**
 *
 * @author Admin
 */
public class Circle01 {
    public static final double PI = 3.14159;

    public static double radiansToDegress(double rads){
        return rads * 100 / PI;
    }

    public double r;

    public double area(){
        return PI * r * r;
    }

    public double circumference(){
        return 2 * PI * r;
    }
}
```

C. Unguided

1. Soal Unguided 1

Sebuah tim mengumpulkan suhu harian suatu kota selama satu minggu (7 hari) dari sensor otomatis. Data ini sudah tersedia sebagai array di `main` sebelum diserahkan untuk diolah. Pada salah satu hari, sensor gagal mengirim data. Hari yang gagal terekam ditandai dengan sebuah nilai penanda pada array, karena suhu asli tidak mungkin sebesar itu. Nilai penanda ini sama untuk semua data suhu yang diolah kapan pun dan oleh siapa pun — bukan sesuatu yang berbeda-beda tiap kali data dibuat — sehingga sebaiknya ditulis satu kali saja sebagai aturan tetap milik class, bukan diketik ulang sebagai angka `-1.0` di setiap method yang membutuhkannya. Buatlah satu class yang bertugas mengolah data suhu ini — memeriksa apakah ada data kosong, mengisinya, dan menghasilkan statistik dasar dari data yang sudah bersih.

Kebutuhan Program

Class ini harus dapat:

1. Menyimpan data suhu 7 hari, dibuat dari array suhu mentah yang sudah ada di `main` (index array mewakili hari).
2. Menyimpan nilai penanda data kosong (`-1.0`) sebagai aturan tetap milik class, yaitu class field yang nilainya tidak dapat diubah dan dipakai bersama oleh semua object.
3. Menampilkan seluruh data apa adanya, menandai hari yang kosong secara jelas.
4. Mencari apakah ada hari yang kosong, dan di index berapa, karena index inilah yang nanti dipakai untuk proses pengisian data.
5. Mengisi hari yang kosong, menggunakan index dari poin 4, dengan nilai perkiraan berdasarkan rata-rata suhu satu hari sebelum dan satu hari sesudahnya.
6. Menghitung rata-rata suhu dari data yang sudah bersih

Jawaban :

PengolahSuhu.java

```
public class PengolahSuhu {  
  
    // Instance field  
    private double[] suhuHarian;  
  
    // Class field
```

```
private static final double NILAI_KOSONG = -1.0;
```

```
// Constructor
```

```
public PengolahSuhu(double[] suhuHarian) {  
    this.suhuHarian = suhuHarian;  
}
```

```
// Menampilkan seluruh data suhu
```

```
public void tampilkanData() {  
    for (int i = 0; i < suhuHarian.length; i++) {  
  
        if (suhuHarian[i] == NILAI_KOSONG) {  
            System.out.println(  
                "Hari " + (i + 1) + " : (kosong)"  
            );  
        } else {  
            System.out.printf(  
                "Hari %d : %.1f\u00B0C%n",  
                (i + 1),  
                suhuHarian[i]  
            );  
        }  
    }  
}
```

```
// Mencari index data yang kosong
```

```
public int cariIndexKosong() {  
    for (int i = 0; i < suhuHarian.length; i++) {
```

```
        if (suhuHarian[i] == NILAI_KOSONG) {  
            return i;  
        }  
    }  
  
    return -1;  
}  
  
// Mengisi data yang kosong  
public void isiDataKosong() {  
    int indexKosong = cariIndexKosong();  
  
    if (indexKosong != -1) {  
        suhuHarian[indexKosong] =  
            (suhuHarian[indexKosong - 1]  
            + suhuHarian[indexKosong + 1]) / 2;  
    }  
}  
  
// Menghitung rata-rata suhu  
public double hitungRataRata() {  
    double total = 0;  
  
    for (double suhu : suhuHarian) {  
        total += suhu;  
    }  
}
```

```
        return total / suhuHarian.length;
    }
}
```

Main.java

```
import java.util.Arrays;

public class Main {

    public static void main(String[] args) {

        /*
         * Mengatur output console menggunakan UTF-8 agar
         * simbol derajat (°) dapat ditampilkan dengan benar.
         */

        System.setOut(new java.io.PrintStream(
            new java.io.FileOutputStream(
                java.io.FileDescriptor.out
            ),
            true,
            java.nio.charset.StandardCharsets.UTF_8
        ));

        // Data suhu 7 hari
        double[] suhuHarian = {
            30.4, 24.3, 26.8, -1.0, 31.4, 30.8, 32.9
        };
    }
}
```

```
// Membuat object PengolahSuhu
PengolahSuhu pengolah = new PengolahSuhu(suhuHarian);

// Menampilkan data awal
System.out.println("=== Data Suhu Awal ===");
pengolah.tampilkanData();

// Mencari index hari yang kosong
int indexKosong = pengolah.cariIndexKosong();

System.out.println();
System.out.println(
    "Index hari kosong (dimulai dari 0): "
    + indexKosong
);

// Mengisi data yang kosong
pengolah.isiDataKosong();

// Menampilkan data setelah pengisian
System.out.println();
System.out.println("=== Data Suhu Setelah Pengisian ===");
pengolah.tampilkanData();

// Menampilkan rata-rata suhu
System.out.printf(
    "%nRata-rata : %.2f\u00B0C%n",
```

```
        pengolah.hitungRataRata()

    );

    // Menampilkan kembali array dari main
    System.out.println();
    System.out.println(
        "Isi array suhuHarian di main setelah "
        + "isiDataKosong() dijalankan:"
    );
    System.out.println(Arrays.toString(suhuHarian));

}

}
```

Output :

```

--- exec:3.6.3:exec (default-cli) @ project-modul-03 ---
=== Data Suhu Awal ===
Hari 1 : 30,4°C
Hari 2 : 24,3°C
Hari 3 : 26,8°C
Hari 4 : (kosong)
Hari 5 : 31,4°C
Hari 6 : 30,8°C
Hari 7 : 32,9°C

Index hari kosong (dimulai dari 0): 3

=== Data Suhu Setelah Pengisian ===
Hari 1 : 30,4°C
Hari 2 : 24,3°C
Hari 3 : 26,8°C
Hari 4 : 29,1°C
Hari 5 : 31,4°C
Hari 6 : 30,8°C
Hari 7 : 32,9°C

Rata-rata : 29,39°C

Isi array suhuHarian di main setelah isiDataKosong() dijalankan:
[30.4, 24.3, 26.8, 29.1, 31.4, 30.8, 32.9]

-----
BUILD SUCCESS
-----

Total time: 2.612 s
Finished at: 2026-10-04T18:38:49+07:00
-----

```

Output

Penjelasan :

Program ini dibuat untuk mengolah data suhu harian selama 7 hari menggunakan konsep Object-Oriented Programming (OOP). Class `PengolahSuhu` menyimpan data suhu sebagai instance field dan nilai `-1.0` sebagai class field untuk menandai data kosong. Program kemudian mencari index data yang kosong, mengisinya berdasarkan rata-rata suhu hari sebelum dan sesudahnya, serta menghitung rata-rata suhu setelah data lengkap. Selain itu, array pada `main` ikut berubah karena object `PengolahSuhu` menggunakan referensi array yang sama.

D. Kesimpulan

Setelah mempelajari konsep abstraksi, enkapsulasi, dan kelas, dapat disimpulkan bahwa ketiganya merupakan bagian penting dalam pemrograman berorientasi objek (OOP). Abstraksi digunakan untuk menentukan bagian penting dari suatu objek, seperti atribut dan perilakunya. Enkapsulasi berfungsi untuk membungkus data dan method dalam sebuah class serta membatasi akses langsung terhadap data. Sementara itu, kelas (class) merupakan rancangan atau cetak biru yang digunakan untuk membuat objek, yang di dalamnya dapat berisi atribut dan method. Dengan memahami ketiga konsep tersebut, program dapat dibuat lebih terstruktur, aman, mudah dipahami, dan mudah dikembangkan.